

Lernskript: Unit Testing in der Praxis (JUnit 5 + Mockito + AssertJ)

Ein Unit Test prüft eine einzelne Klasse vollständig isoliert — ohne Datenbank, ohne Netzwerk, ohne andere echte Abhängigkeiten. Mockito ersetzt diese Abhängigkeiten durch kontrollierte Imitationen ("Mocks").

Teil 1: Die grundlegende Analogie - Das "Warum"

- **Konzept:** Vor jedem Konzert macht die Band einen **Soundcheck** — aber nicht mit echtem Publikum, in einer echten Arena, mit voller Catering-Crew. Der Soundcheck passiert in einem kontrollierten Umfeld: leere Halle, nur die technisch notwendigen Personen.

Wenn der Monitor-Sound falsch ist, weißt du sofort: das Problem liegt beim Monitor-Techniker — nicht am Publikum, nicht am Catering.

Ein **Unit Test** ist dieser Soundcheck. Du testest genau eine Klasse (z.B. `TaskService`) in Isolation. Die Datenbank (`TaskRepository`) wird durch einen **Mock** ersetzt — ein Statist, der auf Kommando genau das zurückgibt, was du ihm sagst.

- **Warum Isolation so wichtig ist:**
 - Wenn ein Test fehlschlägt, weißt du sofort wo das Problem liegt.
 - Tests laufen in Millisekunden — keine Datenbankverbindung nötig.
 - Tests sind wiederholbar: kein Datenbankzustand beeinflusst das Ergebnis.
-

Teil 2: Praktische Anwendungsfälle - Das "Wofür"

Use Case 1: Test-Setup mit Mockito

- **Zweck:** Die Testklasse aufsetzen — Mock erstellen, System unter Test initialisieren.
- **Wichtige Annotationen:**
 - `@ExtendWith(MockitoExtension.class)` : Aktiviert Mockito in JUnit 5.
 - `@Mock` : Erstellt einen Mock (eine leere Imitation der echten Klasse).
 - `@InjectMocks` : Erstellt die zu testende Klasse und injiziert alle Mocks automatisch.
- **Code-Beispiel:**

```
@ExtendWith(MockitoExtension.class)
class TaskServiceTest {

    @Mock
    private TaskRepository taskRepository; // Mock – keine echte DB!

    @InjectMocks
```

```
private TaskService taskService; // System under Test
}
```

Use Case 2: Das AAA-Pattern — Arrange, Act, Assert

- **Zweck:** Struktur für jeden Testfall. Jeder Test folgt exakt diesem Schema.
- **Code-Beispiel:**

```
@Test
@DisplayName("Should map all task entities to DTOs correctly")
void shouldReturnAllTasks() {
    // ARRANGE – Testdaten vorbereiten und Mock programmieren
    Task testTask = new Task();
    testTask.setId(1L);
    testTask.setDescription("Prepare stage");
    testTask.setCompleted(false);
    when(taskRepository.findAll()).thenReturn(List.of(testTask));

    // ACT – Die zu testende Methode aufrufen
    List<TaskDto> result = taskService.getAllTasks();

    // ASSERT – Ergebnis prüfen
    assertThat(result).hasSize(1);
    assertThat(result.get(0).description()).isEqualTo("Prepare stage");
    assertThat(result.get(0).id()).isEqualTo(1L);
}
```

Use Case 3: Stubbing mit `when().thenReturn()`

- **Zweck:** Dem Mock sagen, was er bei einem bestimmten Aufruf zurückgeben soll.
- **Code-Beispiel:**

```
// Stubbing: "Wenn findAll() aufgerufen wird, gib diese Liste zurück"
when(taskRepository.findAll()).thenReturn(List.of(testTask));

// Stubbing: "Wenn save() mit diesem Task aufgerufen wird, gib ihn zurück"
when(taskRepository.save(task)).thenReturn(task);
```

- **Wichtig:** Ohne Stubbing gibt ein Mock für Objekt-Rückgaben `null` zurück, für primitive Typen `0` oder `false`.

Use Case 4: Verhaltens-Prüfung mit `verify()`

- **Zweck:** Prüfen ob eine Methode auf dem Mock aufgerufen wurde — unabhängig vom Rückgabewert. Entscheidend bei `void`-Methoden, die nichts zurückgeben.
- **Code-Beispiel:**

```
@Test
@DisplayName("Should call deleteById when deleteTask is invoked")
void shouldDeleteTask() {
```

```
// Arrange
Long taskId = 1L;

// Act
taskService.deleteTask(taskId);

// Assert – kein assertThat, da void! Nur Verhaltensprüfung.
verify(taskRepository).deleteById(taskId);
}
```

Use Case 5: AssertJ vs. JUnit-Assertions

- **Zweck:** AssertJ bietet flüssige, lesbare Assertions mit besseren Fehlermeldungen.
- **Vergleich:**

```
// JUnit 5 – klassisch, weniger lesbar
assertEquals(1, result.size());
assertEquals("Prepare stage", result.get(0).description());

// AssertJ – modern, flüssig, chainbar
assertThat(result).hasSize(1);
assertThat(result.get(0).description()).isEqualTo("Prepare stage");

// AssertJ – mehrere Prüfungen in einer Kette
assertThat(result)
    .hasSize(1)
    .first()
    .extracting(TaskDto::description)
    .isEqualTo("Prepare stage");
```

Teil 3: Vertiefung (JavaMasta's Profi-Tipps)

- **Mock vs. Stub:** In der Praxis werden beide Begriffe oft synonym verwendet. Technisch: ein **Stub** gibt vordefinierte Werte zurück (`when().thenReturn()`). Ein **Mock** prüft zusätzlich ob bestimmte Methoden aufgerufen wurden (`verify()`). Mockito kann beides.
- **@DisplayName ersetzt Javadoc bei Tests:** Für Testmethoden ist `@DisplayName` die offizielle JUnit 5 Dokumentation. Sie beschreibt in natürlicher Sprache was der Test prüft. Zusätzliches Javadoc auf Testmethoden ist in den meisten professionellen Teams nicht üblich — Klassen-Javadoc bleibt aber Pflicht.
- **when() nicht bei void-Methoden:** `when(mock.voidMethod())` funktioniert nicht. Für void-Methoden die eine Exception werfen sollen, verwendet man stattdessen `doThrow(new Exception()).when(mock).voidMethod()`.
- **AssertJ ist in Spring Boot bereits enthalten:** Die Abhängigkeit `spring-boot-starter-test` bringt AssertJ automatisch mit — kein zusätzlicher Maven-Eintrag nötig. Das ist einer der Gründe, warum AssertJ in Spring-Projekten zum Standard geworden ist.

- **Tests beweisen Architektur:** Ein Test der schwer zu schreiben ist, weist auf ein Architekturproblem hin. Wenn `TaskService` für einen Test eine echte Datenbankverbindung bräuchte, wäre das ein Zeichen dass die Schichten nicht sauber getrennt sind. Gute Tests sind leicht zu schreiben — weil die Architektur stimmt.